

PLC Programming Languages Laboratory Report

Course: Intro to Critical Infrastructure for IT Learners

Lab Title: Lab 02 — PLC Programming Languages

Student Name: Wendell Twumasi

Student ID: C11/26/ACIS/17355

Target VM IP: 192.168.1.10

Date: September 22, 2026

Executive Summary

This laboratory report illustrates the identical functioning of the five standard IEC 61131-3 Programmable Logic Controller (PLC) programming languages, including Ladder Diagram (LD), Function Block Diagram (FBD), Structured Text (ST), Instruction List (IL), and Sequential Function Chart (SFC). Each of the programming languages is utilized within the OpenPLC Editor to operate one standardized motor start/stop holding (seal-in) circuit. Therefore, the document entails the program's variables list, the location of the evidence placeholders, the structural explanation of each programming language, the comparative analysis matrix, as well as the detailed answers to all assessment questions.

1. System Overview & Variable Identification

Industrial automation utilizes standardized tags and uses discrete Boolean data types (BOOL, which evaluate to either 0/FALSE or 1/TRUE). The motor start/stop is controlled by a holding (seal-in) circuit to maintain the motor energized after a momentary Start is released until a momentary Stop is triggered..

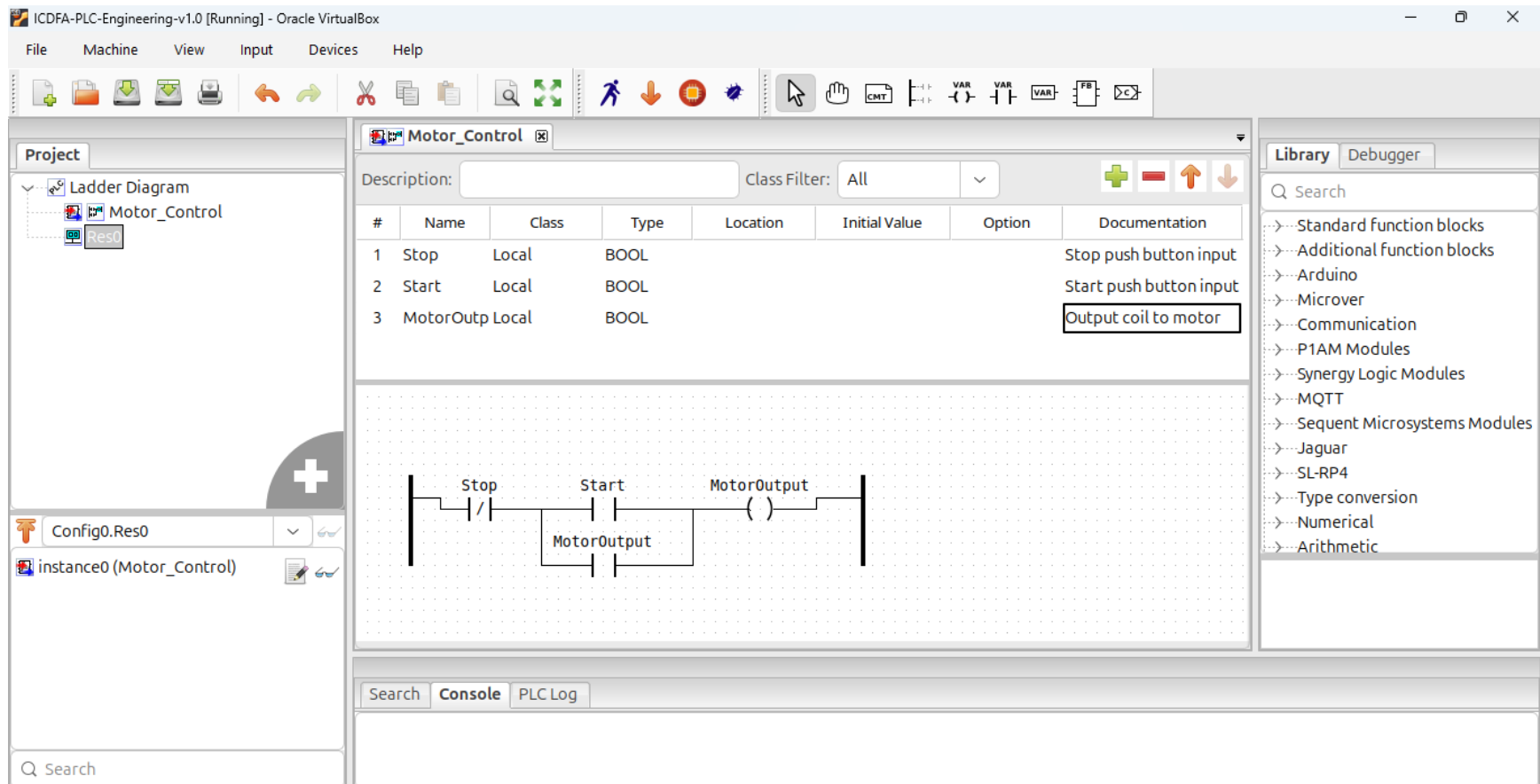
Tag / Variable Table (Step 4 Requirement)

Tag Name	Data Type	Scope / Address	Functional Purpose JPG
Start	BOOL	Local / Input	Momentary normally open push-button input used to initiate the motor run sequence.
Stop	BOOL	Local / Input	Momentary normally closed push-button input used to interrupt power/logic and stop the motor.
MotorOutput (or MotorOutp)	BOOL	Local / Output	Output coil controlling the physical motor starter; provides holding/latching logic feedback.

2. Practical Implementations & Evidence

Evidence Item 1: Ladder Diagram (LD) Implementation

SCREENSHOT 1



(Shows OpenPLC Editor with Motor_Control tab, variable table, and the LD rung containing Start, Stop [Negated], and MotorOutput).

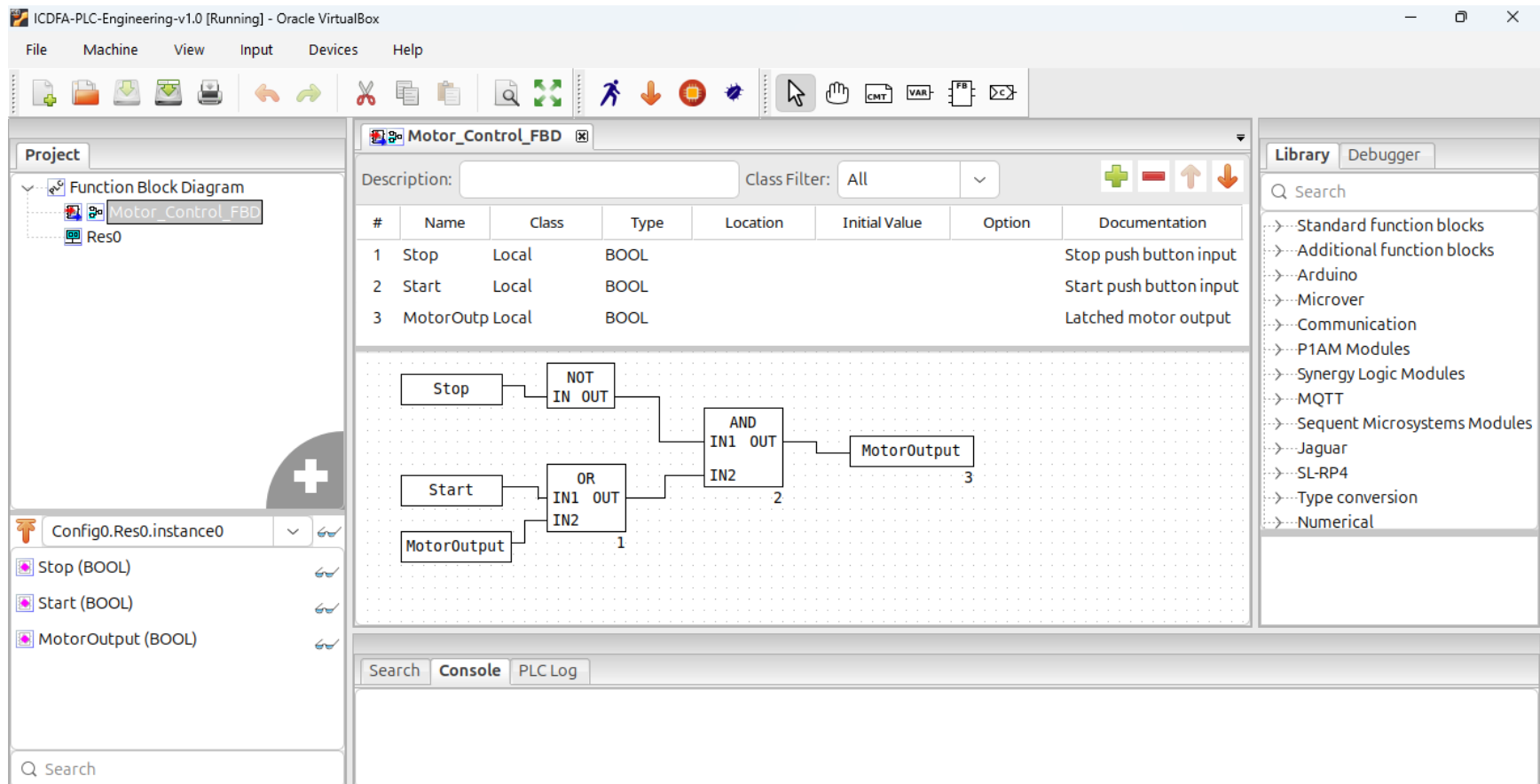
- **Technical Explanation:**

- **Start Contact (| |):** Normally Open contact that closes when Start is activated (TRUE).

- **Stop Contact (| / |):** Negated / Normally Closed contact that allows logic continuity until Stop is pressed (TRUE), which breaks the circuit.
- **MotorOutput Latch Contact (| | in parallel):** Connected in parallel across Start. Once MotorOutput energizes, this contact maintains power flow around Start after it is released.
- **MotorOutput Coil (()):** Output coil energized when a continuous Boolean logic path exists across the rung.

Evidence Item 2: Function Block Diagram (FBD) Implementation

SCREENSHOT 2



(Shows OpenPLC Editor with Motor_Control_FBD tab, variable table, and OR, NOT, AND graphical blocks).

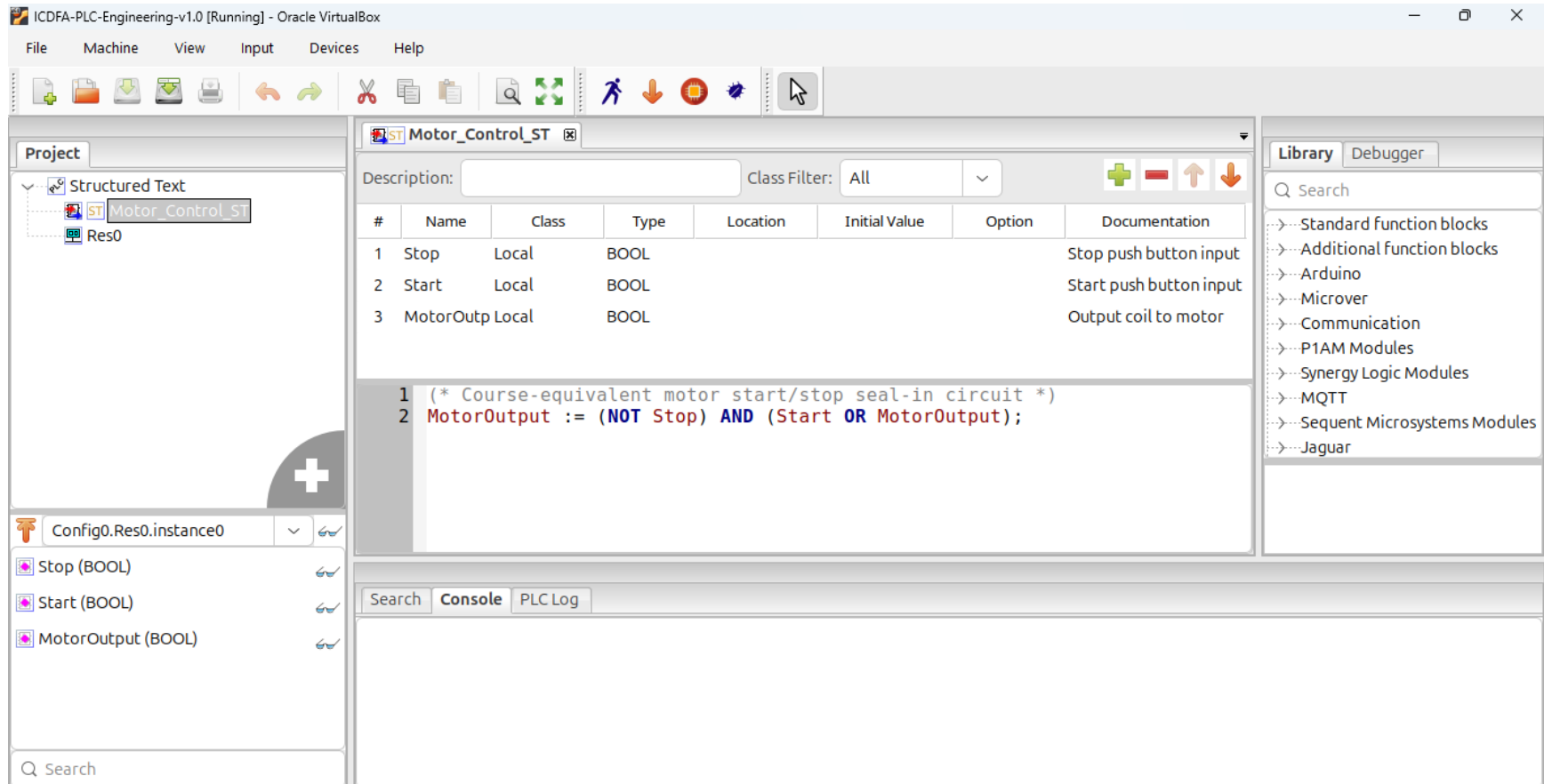
- **Technical Explanation:**

- **OR Block:** Evaluates Start OR MotorOutput, creating the software equivalent of the parallel holding contact.
- **NOT Block:** Inverts the Stop signal (NOT Stop), keeping the interlock high during normal operation.

- **AND Block:** Combines the outputs of the OR block and NOT block ((Start OR MotorOutput) AND NOT Stop) to drive MotorOutput.

Evidence Item 3: Structured Text (ST) Implementation

SCREENSHOT 3



(Shows OpenPLC Editor with Motor_Control_ST tab, variable table, and the textual assignment code).

- **Textual Code Statement:**

Delphi

(* Structured Text Motor Start/Stop Seal-in Circuit *)

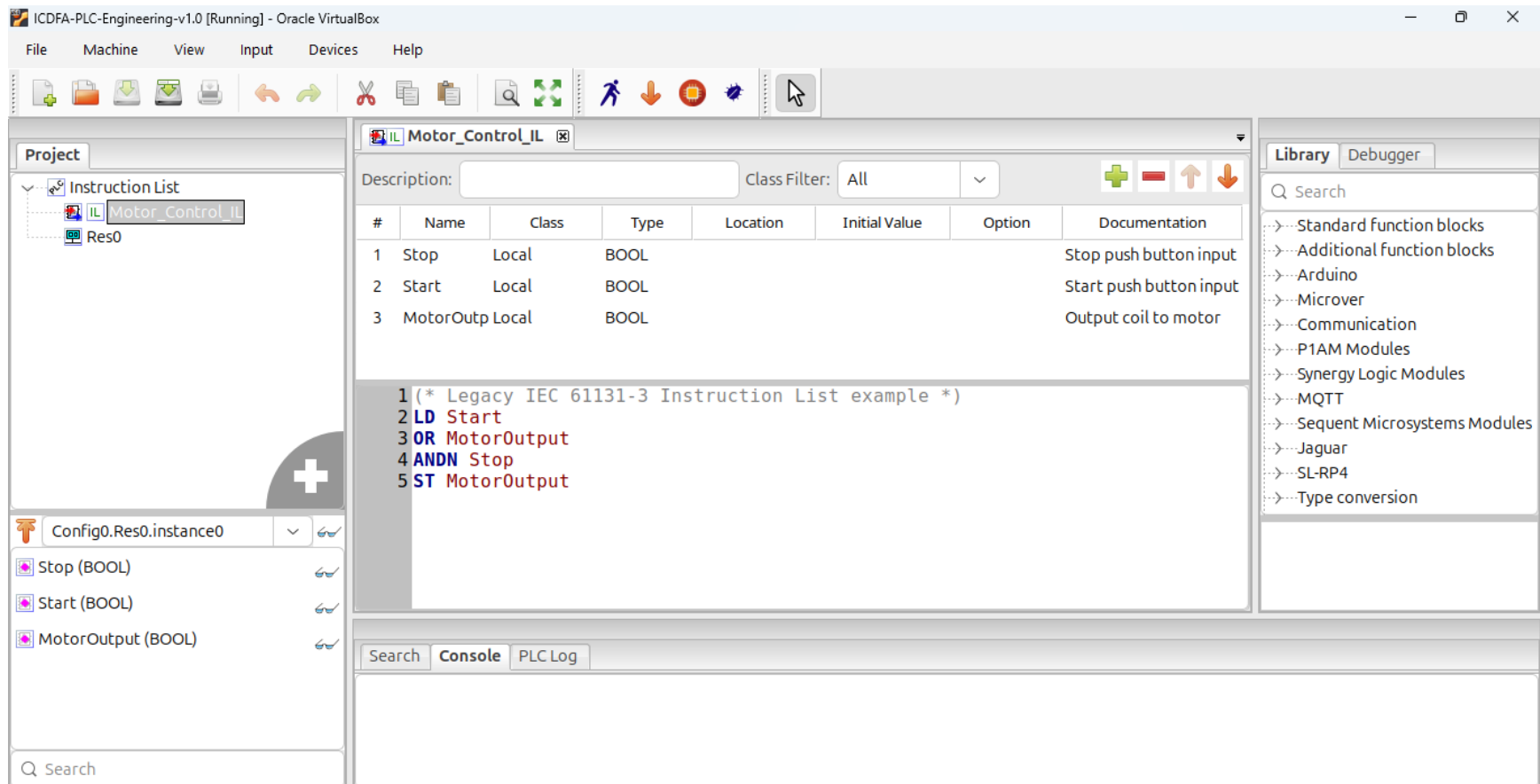
MotorOutput := (NOT Stop) AND (Start OR MotorOutput);

- **Technical Explanation:**

- Uses high-level, procedural syntax evaluated cyclically by the PLC scan engine.
- The expression evaluates whether Stop is unpressed (NOT Stop = TRUE) while either Start is active or MotorOutput is already latched (Start OR MotorOutput).
- The resulting Boolean state is assigned directly to MotorOutput via the assignment operator :=.

Evidence Item 4: Instruction List (IL) Implementation

SCREENSHOT 4



(Shows OpenPLC Editor with Motor_Control_IL tab, variable table, and sequential assembly instructions).

- **Textual Mnemonic Instructions:**

Code snippet

LD Start (* Load Start contact status into accumulator *)

OR MotorOutput (* Perform logical OR with MotorOutput status *)

ANDN Stop (* Perform logical AND NOT with Stop contact *)

ST MotorOutput (* Store result in MotorOutput coil *)

- **Technical Explanation:**

- Low-level assembly-like language operating via an internal result register/accumulator.
- Executes strictly line-by-line: loads the Start state, applies an OR operation with MotorOutput to sustain the seal-in, applies an AND NOT operation with Stop, and stores (ST) the outcome to the output tag.

Evidence Item 5: Sequential Function Chart (SFC) Implementation

SCREENSHOT 5

The screenshot displays the ICDFA-PLC-Engineering-v1.0 [Running] - Oracle VirtualBox window. The main workspace shows the Motor_Control_SFC (Sequential Function Chart) editor. The interface is divided into several panels:

- Project Panel (Left):** Shows the project structure with 'Sequential Function Chart' and 'Motor_Control_SFC' (containing 'Res0').
- Variable Table (Top Center):** A table listing variables for the SFC.

#	Name	Class	Type	Location	Initial Value	Option	Documentation
1	Start	Local	BOOL				Start transition input
2	Stop	Local	BOOL				Stop transition input
3	MotorOutp	Local	BOOL				True while the Running s
- State Transition Diagram (Center):** A diagram showing the SFC logic. It starts with an 'Idle' state (double-bordered box). A transition labeled 'Start AND NOT Stop' leads to a 'Running' state (single-bordered box). A 'Stop' transition leads back to the 'Idle' state.
- Library/Debugger Panel (Right):** Shows the 'Config0.Res0.instance0.*' and '..Start'.

(Shows OpenPLC Editor with Motor_Control_SFC tab, variable table, Idle/Running steps, and transition conditions).

- **Technical Explanation:**

- **Idle Step:** Initial double-bordered step representing the system in its resting state (Motor OFF).
- **Transition 1 (Start AND NOT Stop):** Guard condition that validates the start command and evaluates whether it is safe to transition from Idle to Running.

- **Running Step:** Active operational step associated with the action that energizes MotorOutput := TRUE.
- **Transition 2 (Stop):** Condition that triggers state exit from Running and loops execution back to Idle.

3. Comparative Analysis Matrix (Step 9 Requirement)

Language	Representation Type JPG	Core Strengths JPG	Limitations JPG	Primary Industrial Use Case JPG
Ladder Diagram (LD)	Graphical (Relay Rung Logic)	Intuitive for electrical technicians; easy real-time visual power-flow troubleshooting.	Cluttered and inefficient for complex mathematical calculations or algorithms.	Discrete relay circuits, safety interlocks, motor start/stop controls.
Function Block Diagram (FBD)	Graphical (Signal Flow Blocks)	Excellent visual signal tracking; highly modular and reusable control blocks.	Diagram can become crowded and difficult to follow in large sequential state applications.	Analog process control loops (PID), safety instrumented systems (SIS).
Structured Text (ST)	Textual (High-Level Procedural)	Compact code; ideal for complex math, conditional logic (IF/THEN), loops, and arrays.	Harder for field technicians without software backgrounds to	Complex mathematical algorithms, data parsing, communication

Language	Representation Type JPG	Core Strengths JPG	Limitations JPG	Primary Industrial Use Case JPG
			debug live on physical hardware.	drivers, recipe management.
Instruction List (IL)	Textual (Low-Level Assembly)	High execution speed; low computational and memory footprint.	Difficult to maintain; prone to syntax/logic bugs; deprecated in IEC 61131-3 3rd edition.	Legacy micro-PLCs, execution-critical low-level driver subroutines.
Sequential Function Chart (SFC)	Graphical (State Machine Flowchart)	Clear top-level visual sequencing; decomposes complex multi-stage processes into discrete steps.	Requires underlying languages (LD/ST/FBD) to perform specific step actions.	Multi-stage batch processing, packaging lines, sequential state-machine control.

4. Knowledge Check Responses

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: The ladder diagram is almost identical to the schematic of a real physical electromechanical relay. The diagram uses standard symbols for relay contacts, normally open, and coils with vertical power rails as the real relay schematic. This notation helps maintain diagram consistency and helps maintenance technicians perform visual tracing of the same symbols used in the actual wiring..

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: Structured Text is highly developed in terms of mathematical syntax, variable indexes, arrays, and conditional loops (IF-THEN-ELSE, FOR, WHILE). Whereas LD or FBD programming languages require dozens of blocks or steps associated with complex algebraic calculations or multi-variable logic, only several lines of readable code in ST are needed to solve the same tasks.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: A Sequential Function Chart is preferred for high level state transition based control such as batch processes (i.e. Fill Heat Mix Drain). The SFC isolates each state, transition and action making it much cleaner and less error prone than if you were to try and create that same state transition process in ladder logic using single rungs to represent multiple states and the transitions between them.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: Tags ensure that boolean variables correspond to physical field inputs, internal memory offsets, and hardware output actuators, ensuring consistent control, preventing potentially dangerous physical hardware states, and guaranteeing that conversions between languages do not create unsafe changes in the safety or control of a machine .